

**FORMAN CHRISTIAN COLLEGE
(A CHARTERED UNIVERSITY)**



CSCS 410 A

SP24

Assignment 2

Daim Bin Khalid - 251686775

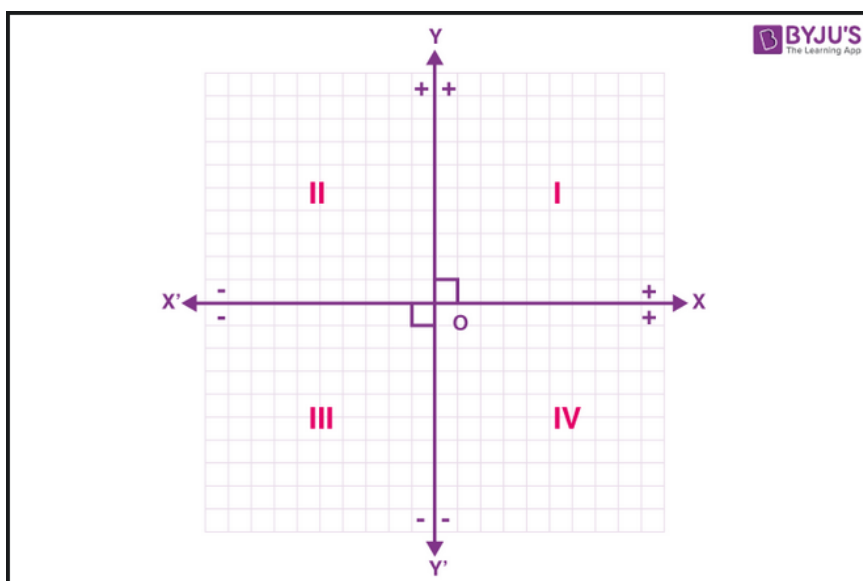
17th May 2024

Part A

Determining sectors for coordinates:

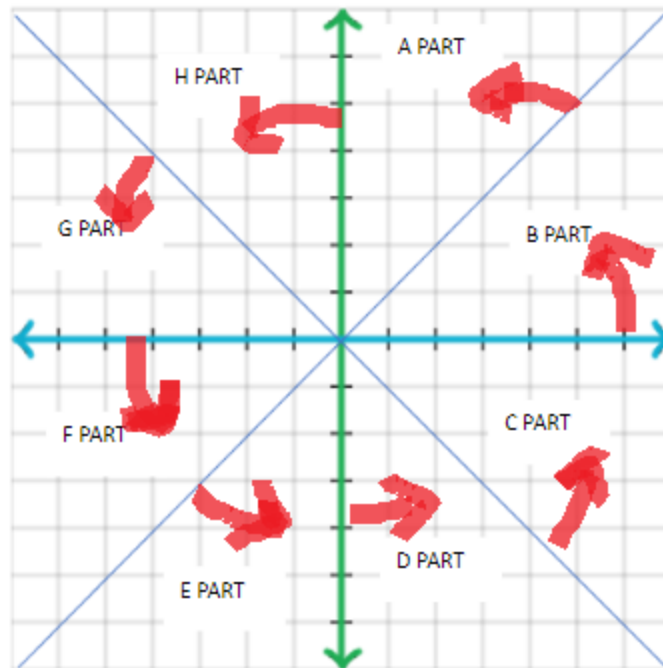
Each point is compared according to its (x, y) coordinates if they x is larger or smaller than y based on 2D plane rules of a regular Cartesian Plane. The four major quadrants are determined in this manner.

- Quadrant $x > 0$, $y > 0$ or (+,+)
- Quadrant $x < 0$, $y > 0$ or (-,+)
- Quadrant $x < 0$, $y < 0$ or (-,-)
- Quadrant $x > 0$, $y < 0$ or (+,-)



Further subdivisions of the four main quadrants were done based on assignment requirements and some test points were dry run in those regions to determine the relation of value of x with y hence making conditions to check coordinates accordingly.

Moreover, if the coordinates lie on the x-axis, y-axis, or on the lines $y = x$ or $y = -x$ then the coordinate belongs to the region on its left (anticlockwise). Origin points do not belong to any sector particularly, so they were added to A for sake of ease.



The following conditions were finally determined and implemented in python code:

```
if (x == 0 and y == 0) or (x > 0 and y > 0 and x <= y):
    sectors["A"] += 1

elif x > 0 and y >= 0 and x > y:
    sectors["B"] += 1

elif x > 0 and y < 0 and x >= abs(y):
    sectors["C"] += 1

elif x >= 0 and y < 0 and abs(y) > x:
    sectors["D"] += 1

elif x < 0 and y < 0 and x >= y:
    sectors["E"] += 1

elif x < 0 and y <= 0 and y > x:
    sectors["F"] += 1

elif x < 0 and y > 0 and abs(x) >= y:
    sectors["G"] += 1

elif x <= 0 and y > 0 and y > abs(x):
    sectors["H"] += 1
```

Sequential:

The sequential program simply loops through the file, reads each line, and checks which region the coordinate lies in.

Output:

```

daimb C:\> ... > parallel > assignment2 > partA
$ python -u "c:\Personal\uni\semester 6\parallel\assignment2\partA\sequential.py"
A: 1321934
B: 1240417
C: 1239339
D: 1238270
E: 1240938
F: 1240802
G: 1238803
H: 1239497
Total: 10000000
Execution time: 6.15948224067688s

```

Parallel:

The parallel program utilizes multiprocessing to read the large file of 10 million points by dividing it into chunks of 10000 lines. Then each of those lines are checked for their regions in the threads.

```

chunk_size = 10000

with open("coordinate_points.txt", "r") as p_file:
    # read file in chunks
    with multiprocessing.Pool(multiprocessing.cpu_count()) as pool:
        sector_counters = pool.map(checkSector, p_file, chunk_size)

```

Output:

```

daimb C:\> ... > parallel > assignment2 > partA
$ python -u "c:\Personal\uni\semester 6\parallel\assignment2\partA\parallel.py"
A: 1321934
B: 1240417
C: 1239339
D: 1238270
E: 1240938
F: 1240802
G: 1238803
H: 1239497
Total: 10000000
Execution time: 5.4708287715911865s

```

Speedup using Amdahl's Law:

Speedup = sequential (without new technique) / parallel (with new technique)

$$\begin{aligned}
 &= 6.16 / 5.47 \\
 &= 1.13x
 \end{aligned}$$

Part B: Finding the Median

Approach:

- Sort using quicksort.
- Get median (middle value based on even or odd elements).

Sequential:

Using quicksort to sort the array and a find median function which handles the even or odd case of elements to get middle value. The value is then floored to make sure it exists in the data set.

```
def partition(array, low, high):
    pivot = array[high]
    i = low - 1

    for j in range(low, high):
        if array[j] <= pivot:
            i = i + 1
            (array[i], array[j]) = (array[j], array[i])

    (array[i + 1], array[high]) = (array[high], array[i + 1])
    return i + 1

def quicksort(array, low, high):
    if low < high:
        pi = partition(array, low, high)
        quicksort(array, low, pi - 1)
        quicksort(array, pi + 1, high)

def find_median(num_list, size):
    if size % 2 == 0:
        return floor((num_list[size // 2 - 1] + num_list[size // 2]) / 2)
    else:
        return floor(num_list[size // 2])
```

Output:

```
daimb C: > ... > parallel > assignment2 > partA
$ python -u "c:\Personal\uni\semester 6\parallel\assignment2\partB\sequential.py"
Median: 49
Execution time: 0.0s
```

Parallel:

Majority of the approach is like sequential except when the quicksort function recursively divides the data and calls itself then the child calls are being done as separate processes to increase efficiency instead of waiting for each arm to finish processing.

```
leftArm = Process(target=quicksort, args=(array, low, pivot - 1))
rightArm = Process(target=quicksort, args=(array, pivot + 1, high))
leftArm.start()
rightArm.start()
leftArm.join()
rightArm.join()
```

Output:

```
daimb C:\> ... > parallel > assignment2 > partA
$ python -u "c:\Personal\uni\semester 6\parallel\assignment2\partB\parallel.py"
Median: 49
Execution time: 0.0s
```

Median of Medians:

This approach uses dividing the data into sorted sub-data arrays then finding their medians. Then of those medians, another median is calculated to be the actual median of the whole data.

```
sub_lists = [arr[j: j + 5] for j in range(0, len(arr), 5)]
medians = [sorted(sublist)[len(sublist) // 2] for sublist in sub_lists]
```

Output:

```
daimb C:\> ... > parallel > assignment2 > partA
$ python -u "c:\Personal\uni\semester 6\parallel\assignment2\partB\medianofmedians.py"
Median: 49
Execution time: 0.11758565902709961s
```

Comparison:

The sequential and parallel approach work at almost the same efficiency but if the dataset were to be massively increased then parallel processing would come more in handy. Meanwhile Median of Medians approach works less efficiently, and its logic makes handling data overly complicated.

References

- <https://byjus.com/maths/cartesian-plane/>
- <https://www.geeksforgeeks.org/quick-sort/>